

Assignment 07 – Page Object Model (POM) Restructure

1. Module Name

02_Unit_Test_Frameworks

2. Experiment Title

Page Object Model (POM) Restructure

3. Problem Statement

Take the working Selenium automation script from Tier 1 and restructure it using the Page Object Model design pattern. The page class should contain locator definitions and UI methods, while the actual test assertions should remain separate from the locators.

4. Objective

The objective of this experiment is to implement the Page Object Model in Selenium Python and separate page-specific elements and actions from the test logic. This improves readability, maintainability, reusability, and organization of the automation code.

5. Tools, Software and Concepts Used

- Python 3
- Selenium WebDriver
- Google Chrome and ChromeDriver
- VS Code
- Page Object Model (POM)
- Python classes and methods
- ID, NAME and XPath locators
- Assertions
- Code reusability and maintainability

6. Source Code / Implementation Steps

1. Open the SauceDemo login page.
2. Create a separate LoginPage class in login_page.py.
3. Define the username, password and login-button locators inside the LoginPage class.
4. Create UI methods for entering the username, entering the password and clicking the login button.
5. Create a reusable login() method that performs the complete login operation.
6. Create test_login.py for the test execution.
7. Import the LoginPage class into the test script.
8. Create the WebDriver and open the login page.
9. Use the Page Object to perform the login operation.
10. Keep the test assertion separate from the page locators and methods.
11. Verify that the resulting URL contains /inventory.html.
12. Keep the browser open temporarily to capture the required screenshots.

7. Output / Labelled Screenshots

The following screenshots are captured during execution and stored in **Source_Code/Screenshots/**:

- Screenshot 1 – SauceDemo Login Page

- Screenshot 2 – Successful Login / Inventory Page
- Screenshot 3 – Successful Execution in Terminal

8. Result

The working Selenium login script was successfully restructured using the Page Object Model. The LoginPage class contains the locators and UI methods, while the test script contains the execution flow and assertion. The login operation completed successfully.

9. Observation

The Page Object Model provides a clear separation between page implementation and test logic. Locators and UI actions are maintained in one place, making the test easier to read and simplifying future maintenance when the web page changes.

10. Conclusion

This experiment provided practical experience in converting a Selenium script into a Page Object Model framework. I learned how to create a page class, define locators, implement reusable UI methods, and keep test assertions separate from page elements. The assignment was completed successfully.